



Computational Geometry

Third Edition

Mark de Berg · Otfried Cheong
Marc van Kreveld · Mark Overmars

Computational Geometry

Algorithms and Applications

Third Edition

Prof. Dr. Mark de Berg
Department of Mathematics
and Computer Science
TU Eindhoven
P.O. Box 513
5600 MB Eindhoven
The Netherlands
mdeberg@win.tue.nl

Dr. Otfried Cheong, né Schwarzkopf
Department of Computer Science
KAIST
Gwangju 335, Yuseong-gu
Daejeon 305-701
Korea
otfried@kaist.edu

Dr. Marc van Kreveld
Department of Information
and Computing Sciences
Utrecht University
P.O. Box 80.089
3508 TB Utrecht
The Netherlands
mare@cs.uu.nl

Prof. Dr. Mark Overmars
Department of Information
and Computing Sciences
Utrecht University
P.O. Box 80.089
3508 TB Utrecht
The Netherlands
markov@cs.uu.nl

ISBN 978-3-540-77973-5

e-ISBN 978-3-540-77974-2

DOI 10.1007/978-3-540-77974-2

ACM Computing Classification (1998): F.2.2, I.3.5

Library of Congress Control Number: 2008921564

© 2008, 2000, 1997 Springer-Verlag Berlin Heidelberg

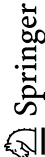
This work is subject to copyright. All rights are reserved, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilm or in any other way, and storage in data banks. Duplication of this publication or parts thereof is permitted only under the provisions of the German Copyright Law of September 9, 1965, in its current version, and permission for use must always be obtained from Springer. Violations are liable for prosecution under the German Copyright Law.

The use of general descriptive names, registered names, trademarks, etc., in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

Cover design: KunkelLopka, Heidelberg

Printed on acid-free paper

9 8 7 6 5 4 3 2 1
springer.com



Preface

Computational geometry emerged from the field of algorithms design and analysis in the late 1970s. It has grown into a recognized discipline with its own journals, conferences, and a large community of active researchers. The success of the field as a research discipline can on the one hand be explained from the beauty of the problems studied and the solutions obtained, and, on the other hand, by the many application domains—computer graphics, geographic information systems (GIS), robotics, and others—in which geometric algorithms play a fundamental role.

For many geometric problems the early algorithmic solutions were either slow or difficult to understand and implement. In recent years a number of new algorithmic techniques have been developed that improved and simplified many of the previous approaches. In this textbook we have tried to make these modern algorithmic solutions accessible to a large audience. The book has been written as a textbook for a course in computational geometry, but it can also be used for self-study.

Structure of the book. Each of the sixteen chapters (except the introductory chapter) starts with a problem arising in one of the application domains. This problem is then transformed into a purely geometric one, which is solved using techniques from computational geometry. The geometric problem and the concepts and techniques needed to solve it are the real topic of each chapter. The choice of the applications was guided by the topics in computational geometry we wanted to cover; they are not meant to provide a good coverage of the application domains. The purpose of the applications is to motivate the reader; the goal of the chapters is not to provide ready-to-use solutions for them. Having said this, we believe that knowledge of computational geometry is important to solve geometric problems in application areas efficiently. We hope that our book will not only raise the interest of people from the algorithms community, but also from people in the application areas.

For most geometric problems treated we give just one solution, even when a number of different solutions exist. In general we have chosen the solution that is easiest to understand and implement. This is not necessarily the most efficient solution. We also took care that the book contains a good mixture of techniques like divide-and-conquer, plane sweep, and randomized algorithms. We decided not to treat all sorts of variations to the problems; we felt it is more important to introduce all main topics in computational geometry than to give more detailed information about a smaller number of topics.

Several chapters contain one or more sections marked with a star. They contain improvements of the solution, extensions, or explain the relation between various problems. They are not essential for understanding the remainder of the book.

Every chapter concludes with a section that is entitled *Notes and Comments*. These sections indicate where the results described in the chapter originated, mention other solutions, generalizations, and improvements, and provide references. They can be skipped, but do contain useful material for those who want to know more about the topic of the chapter.

At the end of each chapter a number of exercises is provided. These range from easy tests to check whether the reader understands the material to more elaborate questions that extend the material covered. Difficult exercises and exercises about starred sections are indicated with a star.

A course outline. Even though the chapters in this book are largely independent, they should preferably not be treated in an arbitrary order. For instance, Chapter 2 introduces plane sweep algorithms, and it is best to read this chapter before any of the other chapters that use this technique. Similarly, Chapter 4 should be read before any other chapter that uses randomized algorithms.

For a first course on computational geometry, we advise treating Chapters 1–10 in the given order. They cover the concepts and techniques that, according to us, should be present in any course on computational geometry. When more material can be covered, a selection can be made from the remaining chapters.

Prerequisites. The book can be used as a textbook for a high-level undergraduate course or a low-level graduate course, depending on the rest of the curriculum. Readers are assumed to have a basic knowledge of the design and analysis of algorithms and data structures; they should be familiar with big-Oh notations and simple algorithmic techniques like sorting, binary search, and balanced search trees. No knowledge of the application domains is required, and hardly any knowledge of geometry. The analysis of the randomized algorithms uses some very elementary probability theory.

Implementations. The algorithms in this book are presented in a pseudo-code that, although rather high-level, is detailed enough to make it relatively easy to implement them. In particular we have tried to indicate how to handle degenerate cases, which are often a source of frustration when it comes to implementing.

We believe that it is very useful to implement one or more of the algorithms; it will give a feeling for the complexity of the algorithms in practice. Each chapter can be seen as a programming project. Depending on the amount of time available one can either just implement the plain geometric algorithms, or implement the application as well.

To implement a geometric algorithm a number of basic data types—points, lines, polygons, and so on—and basic routines that operate on them are needed. Implementing these basic routines in a robust manner is not easy, and takes a lot

of time. Although it is good to do this at least once, it is useful to have a software library available that contains the basic data types and routines. Printers to such libraries can be found on our Web site.

Web site. This book is accompanied by a Web site, which contains a list of errata collected for each edition of the book, all figures and the pseudo code for all algorithms, as well as some other resources. The address is

<http://www.cs.uu.nl/geobook/>

You can also use the address given on our Web site to send us errors you have found, or any other comments you have about the book.

About the third edition. This third edition contains two major additions: In Chapter 7, on Voronoi diagrams, we now also discuss Voronoi diagrams of line segments and farthest-point Voronoi diagrams. In Chapter 12, we have included an extra section on binary space partition trees for low-density scenes, as an introduction to realistic input models. In addition, a large number of small and some larger errors have been corrected (see the list of errata for the second edition on the Web site). We have also updated the notes and comments of every chapter to include references to recent results and recent literature. We have tried not to change the numbering of sections and exercises, so that it should be possible for students in a course to still use the second edition.

Acknowledgments. Writing a textbook is a long process, even with four authors. Many people contributed to the original first edition by providing useful advice on what to put in the book and what not, by reading chapters and suggesting changes, and by finding and correcting errors. Many more provided feedback and found errors in the first two editions. We would like to thank all of them, in particular Pankaj Agarwal, Helmut Alt, Marshall Bern, Jir Bose, Hazel Everett, Gerald Farin, Steve Fortune, Geert-Jan Giezeman, Mordecai Golin, Dan Halperin, Richard Karp, Matthew Katz, Klam Kedem, Nelson Max, Joseph S. B. Mitchell, René van Oostrum, Günter Roie, Henry Shapiro, Sven Skyum, Jack Snoeyink, Gert Vegter, Peter Widmayer, Chee Yap, and Günther Ziegler. We also would like to thank Springer-Verlag for their advice and support during the creation of this book, its new editions, and the translations into other languages (at the time of writing, Japanese, Chinese, and Polish).

Finally we would like to acknowledge the support of the Netherlands' Organization for Scientific Research (N.W.O.) and the Korea Research Foundation (KRF).

January 2008

Mark de Berg
Otfried Cheong
Marc van Kreveld
Mark Overmars

Contents

1 Computational Geometry	1
Introduction	2
1.1 An Example: Convex Hulls	8
1.2 Degeneracies and Robustness	10
1.3 Application Domains	13
1.4 Notes and Comments	15
1.5 Exercises	19
2 Line Segment Intersection	20
Thematic Map Overlay	29
2.1 Line Segment Intersection	33
2.2 The Doubly-Connected Edge List	39
2.3 Computing the Overlay of Two Subdivisions	41
2.4 Boolean Operations	45
2.5 Notes and Comments	46
2.6 Exercises	49
3 Polygon Triangulation	55
Guarding an Art Gallery	59
3.1 Guarding and Triangulations	60
3.2 Partitioning a Polygon into Monotone Pieces	63
3.3 Triangulating a Monotone Polygon	64
3.4 Notes and Comments	66
3.5 Exercises	71
4 Linear Programming	76
Manufacturing with Molds	76
4.1 The Geometry of Casting	76
4.2 Half-Plane Intersection	76
4.3 Incremental Linear Programming	76
4.4 Randomized Linear Programming	76

CONTENTS		CONTENTS			
4.5	Unbounded Linear Programs	79	8.5	Notes and Comments	186
4.6*	Linear Programming in Higher Dimensions	82	8.6	Exercises	188
4.7*	Smallest Enclosing Discs	86			
4.8	Notes and Comments	89			
4.9	Exercises	91			
5	Orthogonal Range Searching	95	9	Delaunay Triangulations	191
	Querying a Database			Height Interpolation	
5.1	1-Dimensional Range Searching	96	9.1	Triangulations of Planar Point Sets	193
5.2	Kd-Trees	99	9.2	The Delaunay Triangulation	196
5.3	Range Trees	105	9.3	Computing the Delaunay Triangulation	199
5.4	Higher-Dimensional Range Trees	109	9.4	The Analysis	205
5.5	General Sets of Points	110	9.5*	A Framework for Randomized Algorithms	208
5.6*	Fractional Cascading	111	9.6	Notes and Comments	214
5.7	Notes and Comments	115	9.7	Exercises	215
5.8	Exercises	117			
6	Point Location	121	10	More Geometric Data Structures	219
	Knowing Where You Are			Windowing	
6.1	Point Location and Trapezoidal Maps	122	10.1	Interval Trees	220
6.2	A Randomized Incremental Algorithm	128	10.2	Priority Search Trees	226
6.3	Dealing with Degenerate Cases	137	10.3	Segment Trees	231
6.4*	A Tail Estimate	140	10.4	Notes and Comments	237
6.5	Notes and Comments	143	10.5	Exercises	239
6.6	Exercises	144			
7	Voronoi Diagrams	147	11	Convex Hulls	243
	The Post Office Problem			Mixing Things	
7.1	Definition and Basic Properties	148	11.1	The Complexity of Convex Hulls in 3-Space	244
7.2	Computing the Voronoi Diagram	151	11.2	Computing Convex Hulls in 3-Space	246
7.3	Voronoi Diagrams of Line Segments	160	11.3*	The Analysis	250
7.4	Farthest-Point Voronoi Diagrams	163	11.4*	Convex Hulls and Half-Space Intersection	253
7.5	Notes and Comments	167	11.5*	Voronoi Diagrams Revisited	254
7.6	Exercises	170	11.6	Notes and Comments	256
			11.7	Exercises	257
8	Arrangements and Duality	173	12	Binary Space Partitions	259
	Supersampling in Ray Tracing			The Painter's Algorithm	
8.1	Computing the Discrepancy	175	12.1	The Definition of BSP Trees	261
8.2	Duality	177	12.2	BSP Trees and the Painter's Algorithm	263
8.3	Arrangements of Lines	179	12.3	Constructing a BSP Tree	264
8.4	Levels and Discrepancy	185	12.4*	The Size of BSP Trees in 3-Space	268
			12.5	BSP Trees for Low-Density Scenes	271
			12.6	Notes and Comments	278
			12.7	Exercises	279
</					

13 Robot Motion Planning**Getting Where You Want to Be**

13.1 Work Space and Configuration Space	283
13.2 A Point Robot	284
13.3 Minkowski Sums	286
13.4 Translational Motion Planning	290
13.5* Motion Planning with Rotations	297
13.6 Notes and Comments	299
13.7 Exercises	303
	305

14 QuadTrees**Non-Uniform Mesh Generation**

14.1 Uniform and Non-Uniform Meshes	307
14.2 QuadTrees for Point Sets	308
14.3 From QuadTrees to Meshes	309
14.4 Notes and Comments	315
14.5 Exercises	318
	320

15 Visibility Graphs**Finding the Shortest Route**

15.1 Shortest Paths for a Point Robot	323
15.2 Computing the Visibility Graph	324
15.3 Shortest Paths for a Translating Polygonal Robot	326
15.4 Notes and Comments	330
15.5 Exercises	331
	332

16 Simplex Range Searching**Windowing Revisited**

16.1 Partition Trees	335
16.2 Multi-Level Partition Trees	336
16.3 Cutting Trees	343
16.4 Notes and Comments	346
16.5 Exercises	352
	353

Bibliography**Index**

	357
	377

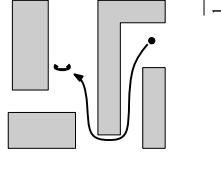
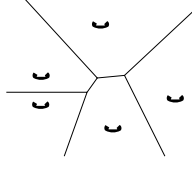
1 Computational Geometry**Introduction**

Imagine you are walking on the campus of a university and suddenly you realize you have to make an urgent phone call. There are many public phones on campus and of course you want to go to the nearest one. But which one is the nearest? It would be helpful to have a map on which you could look up the nearest public phone, wherever on campus you are. The map should show a subdivision of the campus into regions, and for each region indicate the nearest public phone. What would these regions look like? And how could we compute them?

Even though this is not such a terribly important issue, it describes the basics of a fundamental geometric concept, which plays a role in many applications. The subdivision of the campus is a so-called *Voronoi diagram*, and it will be studied in Chapter 7 in this book. It can be used to model trading areas of different cities, to guide robots, and even to describe and simulate the growth of crystals. Computing a geometric structure like a Voronoi diagram requires geometric algorithms. Such algorithms form the topic of this book.

A second example. Assume you located the closest public phone. With a campus map in hand you will probably have little problem in getting to the phone along a reasonably short path, without hitting walls and other objects. But programming a robot to perform the same task is a lot more difficult. Again, the heart of the problem is geometric: given a collection of geometric obstacles, we have to find a short connection between two points, avoiding collisions with the obstacles. Solving this so-called *motion planning* problem is of crucial importance in robotics. Chapters 13 and 15 deal with geometric algorithms required for motion planning.

A third example. Assume you don't have one map but two: one with a description of the various buildings, including the public phones, and one indicating the roads on the campus. To plan a motion to the public phone we have to *overlay* these maps, that is, we have to combine the information in the two maps. Overlaying maps is one of the basic operations of geographic information systems. It involves locating the position of objects from one map in the other, computing the intersection of various features, and so on. Chapter 2 deals with this problem.



These are just three examples of geometric problems requiring carefully designed geometric algorithms for their solution. In the 1970s the field of computational geometry emerged, dealing with such geometric problems. It can be defined as the systematic study of algorithms and data structures for geometric objects, with a focus on exact algorithms that are asymptotically fast. Many researchers were attracted by the challenges posed by the geometric problems. The road from problem formulation to efficient and elegant solutions has often been long, with many difficult and sub-optimal intermediate results. Today there is a rich collection of geometric algorithms that are efficient, and relatively easy to understand and implement.

This book describes the most important notions, techniques, algorithms, and data structures from computational geometry in a way that we hope will be attractive to readers who are interested in applying results from computational geometry. Each chapter is motivated with a real computational problem that requires geometric algorithms for its solution. To show the wide applicability of computational geometry, the problems were taken from various application areas: robotics, computer graphics, CAD/CAM, and geographic information systems.

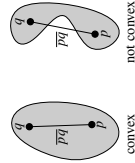
You should not expect ready-to-implement software solutions for major problems in the application areas. Every chapter deals with a single concept in computational geometry; the applications only serve to introduce and motivate the concepts. They also illustrate the process of modeling an engineering problem and finding an exact solution.

1.1 An Example: Convex Hulls

Good solutions to algorithmic problems of a geometric nature are mostly based on two ingredients. One is a thorough understanding of the geometric properties of the problem, the other is a proper application of algorithmic techniques and data structures. If you don't understand the geometry of the problem, all the algorithms of the world won't help you to solve it efficiently. On the other hand, even if you perfectly understand the geometry of the problem, it is hard to solve it effectively if you don't know the right algorithmic techniques. This book will give you a thorough understanding of the most important geometric concepts and algorithmic techniques.

To illustrate the issues that arise in developing a geometric algorithm, this section deals with one of the first problems that was studied in computational geometry: the computation of planar convex hulls. We'll skip the motivation for this problem here; if you are interested you can read the introduction to Chapter 11, where we study convex hulls in 3-dimensional space.

A subset S of the plane is called *convex* if and only if for any pair of points $p, q \in S$ the line segment $\overline{pq}$ is completely contained in S . The *convex hull* $\mathcal{CH}(S)$ of a set S is the smallest convex set that contains S . To be more precise, it is the intersection of all convex sets that contain S .



We will study the problem of computing the convex hull of a finite set P of n points in the plane. We can visualize what the convex hull looks like by a thought experiment. Imagine that the points are nails sticking out of the plane, take an elastic rubber band, hold it around the nails, and let it go. It will snap around the nails, minimizing its length. The area enclosed by the rubber band is the convex hull of P . This leads to an alternative definition of the convex hull of a finite set P of points in the plane: it is the unique convex polygon whose vertices are points from P and that contains all points of P . Of course we should prove rigorously that this is well defined—that is, that the polygon is unique—and that the definition is equivalent to the one given earlier, but let's skip that in this introductory chapter.

How do we compute the convex hull? Before we can answer this question we must ask another question: what does it mean to compute the convex hull? As we have seen, the convex hull of P is a convex polygon. A natural way to represent a polygon is by listing its vertices in clockwise order, starting with an arbitrary one. So the problem we want to solve is this: given a set $P = \{p_1, p_2, \dots, p_n\}$ of points in the plane, compute a list that contains those points from P that are the vertices of $\mathcal{CH}(P)$, listed in clockwise order.

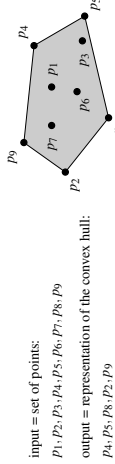


Figure 1.1
Computing a convex hull

The first definition of convex hulls is of little help when we want to design an algorithm to compute the convex hull. It talks about the intersection of all convex sets containing P , of which there are infinitely many. The observation that $\mathcal{CH}(P)$ is a convex polygon is more useful. Let's see what the edges of $\mathcal{CH}(P)$ are. Both endpoints p and q of such an edge are points of P , and if we direct the line through p and q such that $\mathcal{CH}(P)$ lies to the right, then all the points of P must lie to the right of this line. The reverse is also true: if all points of $P \setminus \{p, q\}$ lie to the right of the directed line through p and q , then $\overline{pq}$ is an edge of $\mathcal{CH}(P)$.

Now that we understand the geometry of the problem a little bit better we can develop an algorithm. We will describe it in a style of pseudocode we will use throughout this book.

Algorithm SLOWCONVEXHULL(P)

Input: A set P of points in the plane.

Output: A list L containing the vertices of $\mathcal{CH}(P)$ in clockwise order.

1. $E \leftarrow \emptyset$.
2. **for** all ordered pairs $(p, q) \in P \times P$ with p not equal to q
3. **do** $valid \leftarrow \text{true}$

4. **for** all points $r \in P$ not equal to p or q
5. **do if** r lies to the left of the directed line from p to q
6. **then** $valid \leftarrow \text{false}$.
7. **if** $valid$ **then** Add the directed edge $\vec{pq}$ to E .
8. **From** the set E of edges construct a list L of vertices of $\mathcal{CH}(P)$, sorted in clockwise order.

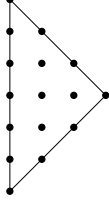
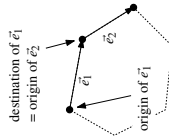
Two steps in the algorithm are perhaps not entirely clear.

The first one is line 5: how do we test whether a point lies to the left or to the right of a directed line? This is one of the primitive operations required in most geometric algorithms. Throughout this book we assume that such operations are available. It is clear that they can be performed in constant time so the actual implementation will not affect the asymptotic running time in order of magnitude. This is not to say that such primitive operations are unimportant or trivial. They are not easy to implement correctly and their implementation will affect the actual running time of the algorithm. Fortunately, software libraries containing such primitive operations are nowadays available. We conclude that we don't have to worry about the test in line 5; we may assume that we have a function available performing the test for us in constant time.

The other step of the algorithm that requires some explanation is the last one. In the loop of lines 2–7 we determine the set E of convex hull edges. From E we can construct the list L as follows. The edges in E are directed, so we can speak about the origin and the destination of an edge. Because the edges are directed such that the other points lie to their right, the destination of an edge comes after its origin when the vertices are listed in clockwise order. Now remove an arbitrary edge $\vec{e}_1$ from E . Put the origin of $\vec{e}_1$ as the first point into L , and the destination as the second point. Find the edge $\vec{e}_2$ in E whose origin is the destination of $\vec{e}_1$, remove it from E , and append its destination to L . Next, find the edge $\vec{e}_3$ whose origin is the destination of $\vec{e}_2$, remove it from E , and append its destination to L . We continue in this manner until there is only one edge left in E . Then we are done; the destination of the remaining edge is necessarily the origin of $\vec{e}_1$, which is already the first point in L . A simple implementation of this procedure takes $O(n^2)$ time. This can easily be improved to $O(n \log n)$, but the time required for the rest of the algorithm dominates the total running time anyway.

Analyzing the time complexity of SLOWCONVEXHULL is easy. We check $n^2 - n$ pairs of points. For each pair we look at the $n - 2$ other points to see whether they all lie on the right side. This will take $O(n^3)$ time in total. The final step takes $O(n^2)$ time, so the total running time is $O(n^3)$. An algorithm with a cubic running time is too slow to be of practical use for anything but the smallest input sets. The problem is that we did not use any clever algorithmic design techniques; we just translated the geometric insight into an algorithm in a brute-force manner. But before we try to do better, it is useful to make several observations about this algorithm.

We have been a bit careless when deriving the criterion of when a pair p, q defines an edge of $\mathcal{CH}(P)$. A point r does not always lie to the right or to the



left of the line through p and q , it can also happen that it lies *on* this line. What should we do then? This is what we call a *degenerate case*, or a *degeneracy* for short. We prefer to ignore such situations when we first think about a problem, so that we don't get confused when we try to figure out the geometric properties of a problem. However, these situations do arise in practice. For instance, if we create the points by clicking on a screen with a mouse, all points will have small integer coordinates, and it is quite likely that we will create three points on a line.

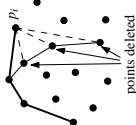
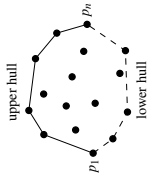
To make the algorithm correct in the presence of degeneracies we must reformulate the criterion above as follows: a directed edge $\vec{pq}$ is an edge of $\mathcal{CH}(P)$ if and only if all other points $r \in P$ lie either strictly to the right of the directed line through p and q , or they lie on the open line segment $\vec{pq}$. (We assume that there are no coinciding points in P .) So we have to replace line 5 of the algorithm by this more complicated test.

We have been ignoring another important issue that can influence the correctness of the result of our algorithm. We implicitly assumed that we can somehow test exactly whether a point lies to the right or to the left of a given line. This is not necessarily true: if the points are given in floating point coordinates and the computations are done using floating point arithmetic, then there will be rounding errors that may distort the outcome of tests.

Imagine that there are three points p, q , and r , that are nearly collinear, and that all other points lie far to the right of them. Our algorithm tests the pairs (p, q) , (r, q) , and (p, r) . Since these points are nearly collinear, it is possible that the rounding errors lead us to decide that r lies to the right of the line from p to q , that p lies to the right of the line from r to q , and that q lies to the right of the line from p to r . Of course this is geometrically impossible—but the floating point arithmetic doesn't know that! In this case the algorithm will accept all three edges. Even worse, all three tests could give the opposite answer, in which case the algorithm rejects all three edges, leading to a gap in the boundary of the convex hull. And this leads to a serious problem when we try to construct the sorted list of convex hull vertices in the last step of our algorithm. This step assumes that there is exactly one edge starting in every convex hull vertex, and exactly one edge ending there. Due to the rounding errors there can suddenly be two, or no, edges starting in vertex p . This can cause the program implementing our simple algorithm to crash, since the last step has not been designed to deal with such inconsistent data.

Although we have proven the algorithm to be correct and to handle all special cases, it is not *robust*: small errors in the computations can make it fail in completely unexpected ways. The problem is that we have proven the correctness assuming that we can compute exactly with real numbers.

We have designed our first geometric algorithm. It computes the convex hull of a set of points in the plane. However, it is quite slow—its running time is $O(n^3)$ —it deals with degenerate cases in an awkward way, and it is not robust. We should try to do better.



To this end we apply a standard algorithmic design technique: we will develop an *incremental algorithm*. This means that we will add the points in P one by one, updating our solution after each addition. We give this incremental approach a geometric flavor by adding the points from left to right. So we first sort the points by x -coordinate, obtaining a sorted sequence $p_1, \dots, p_n$, and then we add them in that order. Because we are working from left to right, it would be convenient if the convex hull vertices were also ordered from left to right as they occur along the boundary. But this is not the case. Therefore we first compute only those convex hull vertices that lie on the *upper hull*, which is the part of the convex hull running from the leftmost point p_1 to the rightmost point p_n when the vertices are listed in clockwise order. In other words, the upper hull contains the convex hull edges bounding the convex hull from above. In a second scan, which is performed from right to left, we compute the remaining part of the convex hull, the *lower hull*.

The basic step in the incremental algorithm is the update of the upper hull after adding a point p_i . In other words, given the upper hull of the points $p_1, \dots, p_{i-1}$, we have to compute the upper hull of $p_1, \dots, p_i$. This can be done as follows. When we walk around the boundary of a polygon in clockwise order, we make a turn at every vertex. For an arbitrary polygon this can be both a right turn and a left turn, but for a convex polygon every turn must be a right turn. This suggests handling the addition of p_i in the following way. Let L_{upper} be a list that stores the upper vertices in left-to-right order. We first append p_i to L_{upper} . This is correct because p_i is the rightmost point of the ones added so far, so it must be on the upper hull. Next, we check whether the last three points in L_{upper} make a right turn. If this is the case there is nothing more to do; L_{upper} contains the vertices of the upper hull of $p_1, \dots, p_i$, and we can proceed to the next point, p_{i+1} . But if the last three points make a left turn, we have to delete the middle one from the upper hull. In this case we are not finished yet: it could be that the new last three points still do not make a right turn, in which case we again have to delete the middle one. We continue in this manner until the last three points make a right turn, or until there are only two points left.

We now give the algorithm in pseudocode. The pseudocode computes both the upper hull and the lower hull. The latter is done by treating the points from right to left, analogous to the computation of the upper hull.

Algorithm CONVEXHULL(P)

Input. A set P of points in the plane.

Output. A list containing the vertices of $\mathcal{CH}(P)$ in clockwise order.

1. Sort the points by x -coordinate, resulting in a sequence $p_1, \dots, p_n$.
2. Put the points p_1 and p_2 in a list L_{upper} , with p_1 as the first point.
3. **for** $i \leftarrow 3$ **to** n
4. **do** Append p_i to L_{upper} .
5. **while** L_{upper} contains more than two points **and** the last three points in L_{upper} do not make a right turn
6. **do** Delete the middle of the last three points from L_{upper} .
7. Put the points p_n and p_{n-1} in a list L_{lower} , with p_n as the first point.

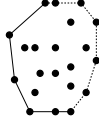
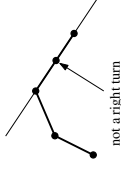
8. **for** $i \leftarrow n-2$ **down to** 1
9. **do** Append p_i to L_{lower} .
10. **while** L_{lower} contains more than 2 points **and** the last three points in L_{lower} do not make a right turn
11. **do** Delete the middle of the last three points from L_{lower} .
12. Remove the first and the last point from L_{lower} to avoid duplication of the points where the upper and lower hull meet.
13. Append L_{lower} to L_{upper} , and call the resulting list $\mathcal{L}$.
14. **return** $\mathcal{L}$

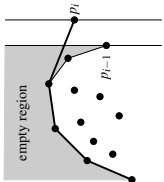
Once again, when we look closer we realize that the above algorithm is not correct. Without mentioning it, we made the assumption that no two points have the same x -coordinate. If this assumption is not valid the order on x -coordinate is not well defined. Fortunately, this turns out not to be a serious problem. We only have to generalize the ordering in a suitable way: rather than using only the x -coordinate of the points to define the order, we use the lexicographic order. This means that we first sort by x -coordinate, and if points have the same x -coordinate we sort them by y -coordinate.

Another special case we have ignored is that the three points for which we have to determine whether they make a left or a right turn lie on a straight line. In this case the middle point should not occur on the convex hull, so collinear points must be treated as if they make a left turn. In other words, we should use a test that returns true if the three points make a right turn, and false otherwise. (Note that this is simpler than the test required in the previous algorithm when there were collinear points.)

With these modifications the algorithm correctly computes the convex hull: the first scan computes the upper hull, which is now defined as the part of the convex hull running from the lexicographically smallest vertex to the lexicographically largest vertex, and the second scan computes the remaining part of the convex hull.

What does our algorithm do in the presence of rounding errors in the floating point arithmetic? When such errors occur, it can happen that a point is removed from the convex hull although it should be there, or that a point inside the real convex hull is not removed. But the structural integrity of the algorithm is unharmed: it will compute a closed polygonal chain. After all, the output is a list of points that we can interpret as the clockwise listing of the vertices of a polygon, and any three consecutive points form a right turn or, because of the rounding errors, they almost form a right turn. Moreover, no point in P can be far outside the computed hull. The only problem that can still occur is that, when three points lie very close together, a turn that is actually a sharp left turn can be interpreted as a right turn. This might result in a dent in the resulting polygon. A way out of this is to make sure that points in the input that are very close together are considered as being the same point, for example by rounding. Hence, although the result need not be exactly correct—but then, we cannot hope for an exact result if we use inexact arithmetic—it does make sense. For many applications this is good enough. Still, it is wise to be careful in the implementation of the basic test to avoid errors as much as possible.





We conclude with the following theorem:

Theorem 1.1 *The convex hull of a set of n points in the plane can be computed in $O(n \log n)$ time.*

Proof. We will prove the correctness of the computation of the upper hull; the lower hull computation can be proved correct using similar arguments. The proof is by induction on the number of point treated. Before the **for**-loop starts, the list L_{upper} contains the points p_1 and p_2 , which trivially form the upper hull of $\{p_1, p_2\}$. Now suppose that L_{upper} contains the upper hull vertices of $\{p_1, \dots, p_{i-1}\}$ and consider the addition of p_i . After the execution of the **while**-loop and because of the induction hypothesis, we know that the points in L_{upper} form a chain that only makes right turns. Moreover, the chain starts at the lexicographically smallest point of $\{p_1, \dots, p_i\}$ and ends at the lexicographically largest point, namely p_i . If we can show that all points of $\{p_1, \dots, p_i\}$ that are not in L_{upper} are below the chain, then L_{upper} contains the correct points. By induction we know there is no point above the chain that we had before p_i was added. Since the old chain lies below the new chain, the only possibility for a point to lie above the new chain is if it lies in the vertical slab between p_{i-1} and p_i . But this is not possible, since such a point would be in between p_{i-1} and p_i in the lexicographical order. (You should verify that a similar argument holds if p_{i-1} and p_i or any other points, have the same x -coordinate.)

To prove the time bound, we note that sorting the points lexicographically can be done in $O(n \log n)$ time. Now consider the computation of the upper hull. The **for**-loop is executed a linear number of times. The question that remains is how often the **while**-loop inside it is executed. For each execution of the **for**-loop the **while**-loop is executed at least once. For any extra execution a point is deleted from the current hull. As each point can be deleted only once during the construction of the upper hull, the total number of extra executions over all **for**-loops is bounded by n . Similarly, the computation of the lower hull takes $O(n)$ time. Due to the sorting step, the total time required for computing the convex hull is $O(n \log n)$. $\square$

The final convex hull algorithm is simple to describe and easy to implement. It only requires lexicographic sorting and a test whether three consecutive points make a right turn. From the original definition of the problem it was far from obvious that such an easy and efficient solution would exist.

1.2 Degeneracies and Robustness

As we have seen in the previous section, the development of a geometric algorithm often goes through three phases.

In the first phase, we try to ignore everything that will clutter our understanding of the geometric concepts we are dealing with. Sometimes collinear points are a nuisance, sometimes vertical line segments are. When first trying to design or understand an algorithm, it is often helpful to ignore these degenerate cases.